SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192511009**
Name	**Avishek N**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I _Avishek N (192511009)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Question 1 — AI-Powered Emergency Drone Delivery

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic — roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Task 1: Behaviour of Greedy Best-First Search

Greedy Best-First Search (GBFS) expands the node that appears closest to the goal purely on the basis of the heuristic function h(n), the straight-line distance to the delivery point, while completely ignoring the actual path cost g(n) incurred so far.

Strengths: GBFS is fast and consumes less memory because it commits early to nodes that look promising, which suits the drone's need for quick decisions under urgency. It works well when the heuristic reliably reflects real obstacles.

Weaknesses under uncertainty: Because GBFS never accounts for accumulated cost, it can be lured toward a route that looks close in straight-line distance but is actually blocked by floodwater or has a very high terrain/weather cost, forcing expensive detours. It is also neither complete nor optimal — in a dynamic flood environment, it may loop, choose a locally attractive but globally unsafe route, or fail to find a path if the greedy choice repeatedly leads into blocked regions.

Task 2: Behaviour of A* Search

A* Search evaluates every candidate node using f(n) = g(n) + h(n), where g(n) is the actual cost accumulated from the start (capturing real terrain and weather penalties) and h(n) is the straight-line-distance heuristic to the goal.

Balancing cost and heuristic: By combining both terms, A* refuses to commit to a route only because it looks close on the map; it only prefers a node if it is genuinely cheaper overall. This means A* naturally avoids a seemingly-close but weather-battered path in favour of a slightly longer but safer and cheaper one, as long as the heuristic is admissible (never overestimates the true remaining cost).

Result: A* guarantees an optimal, minimum-cost path when h(n) is admissible and consistent, which is essential for an emergency-medicine drone where minimising delay and risk together, not just distance, is the real objective.

Task 3: Impact of Heuristic Accuracy

Aspect	Greedy Best-First Search	A* Search
Accurate heuristic	Performs very fast and often finds a good route, but optimality is still not guaranteed.	Performs fast AND optimal; fewer nodes are expanded because the heuristic prunes the search effectively.
Inaccurate / misleading heuristic	Can be badly misled, expanding many nodes into blocked or high-cost regions; solution quality degrades sharply.	Still finds the optimal path as long as h(n) remains admissible, though search may expand more nodes and slow down.
Overestimating heuristic	Increases risk of choosing unsafe short-looking routes.	Optimality guarantee is lost if h(n) overestimates true cost.

In short, heuristic accuracy is the single biggest factor governing GBFS quality, whereas A* is more robust: an accurate heuristic mainly improves A*'s speed rather than its correctness.

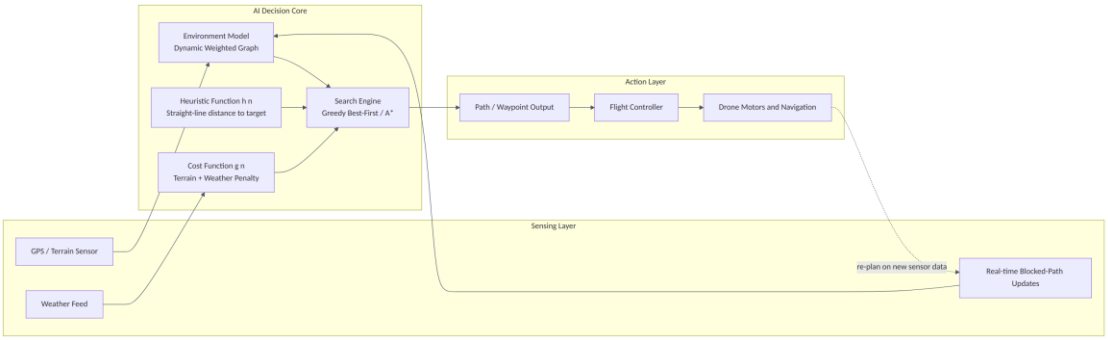
Task 4: Effect of Dynamic Changes (Blocked Paths)

- Both algorithms operate on a snapshot of the graph; when a road suddenly becomes blocked, previously computed paths and cost/heuristic values become stale and the drone must re-plan.
- GBFS re-planning is cheap computationally but risks repeating the same mistake — greedily heading toward the same blocked-looking shortcut again.
- A* re-planning (e.g., via Repeated A* / D* Lite style incremental replanning) is more expensive per cycle because it maintains g-values, but produces a consistently reliable route even as the graph changes.
- Frequent dynamic changes increase the number of re-planning cycles for both algorithms, so real systems typically use time-bounded or incremental variants (D*, LPA*, Anytime A*) to keep response times acceptable.

Task 5: Recommendation

Recommended algorithm: A* Search (preferably an incremental/real-time variant such as D* Lite) is the most suitable choice for this scenario.

Justification: A* offers the optimality and completeness that GBFS lacks, which matters when the drone is carrying life-saving medicine and cannot afford an unsafe or excessively costly route. Its use of g(n) makes it naturally weather- and terrain-aware, and modern incremental A* variants can re-plan quickly enough to satisfy real-time constraints, giving the best balance of optimality, safety, and responsiveness.



Question 2 — Smart City Traffic Signal Optimization

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections, minimising total waiting time, fuel consumption, and traffic congestion, while continuously adjusting to changing traffic flow. The search space is extremely large, patterns change dynamically, and the system may get stuck in locally optimal solutions.

Task 1: Problem Formulation & Why Traditional Search is Inefficient

Formulation: This is an optimization problem, not a path-finding problem. The state is a configuration vector of signal timings across all intersections (green/red durations, offsets, cycle lengths). The objective function combines total waiting time, fuel consumption, and congestion into a single cost to minimise, subject to constraints such as minimum green time and pedestrian-safety intervals.

Why traditional search fails: Classical uninformed/informed search (BFS, DFS, A*) explores explicit paths toward a single goal state, but here there is no single 'goal state' to search toward — instead, we need the best point in a continuous, extremely high-dimensional configuration space. The branching factor across hundreds of intersections makes the state space combinatorially explosive, so systematic path-search techniques become computationally infeasible; local/optimization search is required instead.

Task 2: Hill Climbing in this Scenario

Hill Climbing starts from an initial timing configuration and repeatedly moves to a neighbouring configuration (e.g., tweak one intersection's green duration) if it improves the objective function, stopping when no neighbour improves further.

Limitations: It is a purely local method with no backtracking or lookahead, so it easily settles for a locally good but globally poor timing plan. It has no mechanism to escape once it reaches a point where every immediate neighbour is worse or equal, which is highly likely across hundreds of interacting intersections.

Task 3: Local Maxima, Plateaus and Ridges

Issue	Real-World Interpretation
Local Maxima	A timing plan where every small tweak to one intersection makes things worse locally, even though a completely different combination of settings across several intersections together would perform far better city-wide.
Plateaus	Adjusting a signal by a few seconds produces no measurable change in waiting time/fuel because traffic volume at that time is roughly constant, so Hill Climbing cannot tell which direction to move and may wander or stall.
Ridges	Improvement requires simultaneously changing two or more correlated intersections together (e.g., coordinated 'green wave' timing along a corridor); moving only one at a time never crosses the ridge even though a combined move would clearly help.

Task 4: Overcoming these Limitations

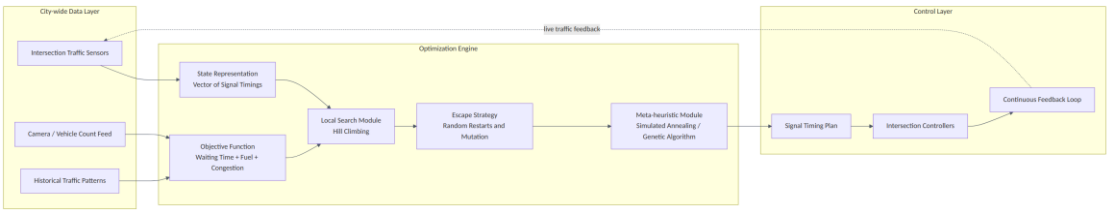
Simulated Annealing (SA): SA occasionally accepts a worse configuration with probability e^(-Δ/T), where Δ is the increase in cost and T is a temperature parameter that gradually decreases. This controlled randomness lets the search escape local maxima and cross plateaus early on, while converging to a near-optimal, stable solution as T approaches zero.

Genetic Algorithms (GA): GA maintains a population of many candidate timing plans simultaneously, combining (crossover) and randomly perturbing (mutation) the fittest plans across generations. Because it searches many regions of the space in parallel rather than following a single trajectory, GA is naturally resistant to getting trapped in one local maximum and can discover coordinated multi-intersection improvements that ridge-shaped landscapes require.

Task 5: Recommendation

Recommended approach: A Genetic Algorithm, optionally hybridised with Simulated Annealing for local fine-tuning of the best individuals, is best suited for city-wide deployment.

Justification: GA scales well because intersections can be encoded as independent genes and evaluated in parallel across a large population, matching the scale of hundreds of intersections. It adapts continuously by re-running with fresh traffic data as new generations, giving the adaptability needed for constantly changing traffic patterns, while its population diversity directly counters local maxima, plateaus and ridges better than a single-trajectory method like plain Hill Climbing or even standalone Simulated Annealing.



Question 3 — Autonomous Mars Rover Exploration

An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing, and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Task 1: Why an Online Search Agent is Required

Offline search assumes the agent has complete, accurate knowledge of the environment before it starts acting and can compute an entire solution path in advance. The Mars rover has no such map — the terrain is discovered only as it physically moves through it, and communication delay to Earth (several minutes one way) makes constant human replanning impossible. An online search agent interleaves computation with action: it senses, decides one step (or a short lookahead), acts, and only then re-plans based on what it actually observed, which matches the rover's need to operate in unknown terrain in real time.

Task 2: Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover only perceives terrain within sensor range, so it cannot guarantee that a chosen path is truly safe beyond the visible horizon, and previously 'safe-looking' cells may turn out to be hazardous once reached.
- Irreversible actions: some rover moves (e.g., descending a slope) cannot easily be undone, so the agent must make cautious/pessimistic decisions when it is uncertain rather than exploring purely optimistically.
- Dynamic and costly re-sensing: dust storms and lighting changes affect sensor reliability, and every sensing/compute cycle consumes limited power and time budget, which online algorithms must respect.
- No admissible global heuristic is available in advance since the terrain is unmapped, so heuristic estimates must be learned/updated on the fly.

Task 3: Building and Updating the Internal Model

The rover maintains a local occupancy/knowledge map that starts mostly unknown. At every step, sensor data (stereo cameras, LIDAR) is used to mark nearby cells as free, occupied (rock/crater), or still-unknown. Algorithms such as Learning Real-Time A* (LRTA*) additionally maintain and update a heuristic-value table H(s) for visited states: whenever the rover discovers that a state's true cost-to-goal is higher than previously estimated, it revises H(s) upward so the same mistake is not repeated. Over repeated exploration passes, this progressively 'learns' the terrain, letting later decisions become more informed than earlier ones.

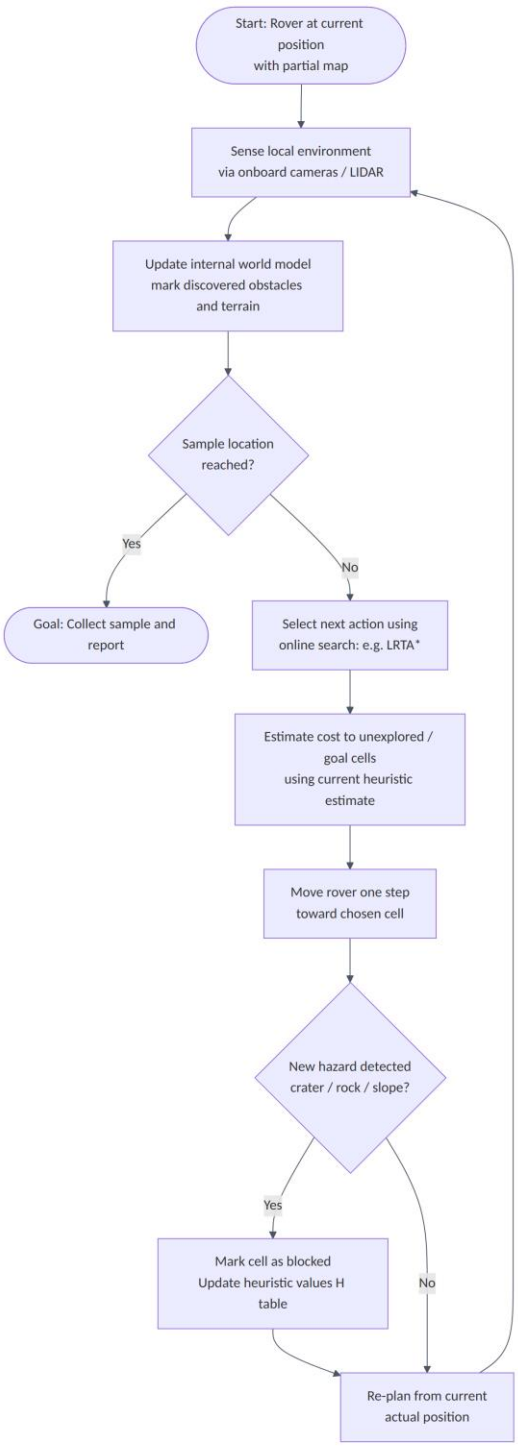
Task 4: Online Search vs Uninformed/Informed Offline Search

Property	Uninformed Search (e.g., BFS)	Informed Offline Search (e.g., A*)	Online Search (e.g., LRTA*)
Map requirement	Full map needed in advance	Full map + heuristic needed in advance	No map required; builds map as it explores
Adaptability to new hazards	None — plan is fixed	None — plan is fixed once computed	High — re-plans immediately from new percepts
Computation pattern	All computed before acting	All computed before acting	Interleaved: sense → decide → act → repeat
Suitability for Mars rover	Not suitable (no map exists)	Not suitable (no map exists)	Suitable — matches real operating conditions

Task 5: Justification of the Most Suitable Approach

Recommended approach: An online, informed search agent such as LRTA* (or a Real-Time Dynamic Programming variant), combined with a locally-built occupancy grid, is most suitable.

Justification: It directly addresses uncertainty by only trusting what has actually been sensed and revising beliefs as new hazards appear, supports adaptability because every step naturally incorporates fresh sensor data without waiting for a full re-plan from scratch, and improves safety since decisions are always grounded in the rover's real, currently-known surroundings rather than a stale or incomplete global plan.



Question 4 — University Exam Timetable Generation

A university is designing an AI system to automatically generate exam timetables satisfying multiple constraints: no overlapping exams for any student, a limited number of exam halls, certain subjects scheduled before others, faculty availability, and special accommodations, with complexity increasing as courses and students grow.

Task 1: Formulating the Problem as a CSP

The exam scheduling problem maps naturally onto a Constraint Satisfaction Problem: each exam to be scheduled is a variable, its possible (time slot, hall) combinations form its domain, and every rule in the scenario (no overlaps, hall capacity, precedence, faculty availability, accommodations) becomes a constraint that any valid assignment of variables to domain values must satisfy simultaneously.

Task 2: Variables, Domains and Constraints

Element	Definition in this Scenario
Variables	One variable per course exam, e.g. Exam_DBMS, Exam_AI, Exam_OS, ...
Domains	For each exam: all valid (time-slot, hall) pairs, e.g. {(Mon-9AM, Hall-1), (Mon-9AM, Hall-2), (Tue-2PM, Hall-1), ...}
Unary Constraints	An exam requiring a special accommodation room can only take domain values that include an accessible hall.
Binary Constraints	No two exams sharing a common student may be assigned the same time slot (Exam_i.slot ≠ Exam_j.slot when they share students).
Resource Constraints	Total exams assigned to a hall+slot combination cannot exceed hall capacity.
Precedence Constraints	Exam_A.slot must occur strictly before Exam_B.slot when subject A is a prerequisite for B.
Availability Constraints	Exam.slot must be within the invigilating faculty's declared available slots.

Task 3: How Backtracking Search Works Here

Backtracking search assigns exams one at a time (commonly choosing the most-constrained exam first via the Minimum Remaining Values heuristic), picks a (slot, hall) value from its domain, and checks it against all constraints already assigned so far. If the value satisfies every relevant constraint, the search moves on to the next exam; if it violates a constraint, the algorithm tries the next candidate value; if no value in the domain works, the search backtracks — undoing the previous exam's assignment and trying its next alternative. This continues (with depth-first exploration) until either every exam is validly scheduled or the search proves no valid timetable exists under the current constraints.

Task 4: How Forward Checking Improves Efficiency

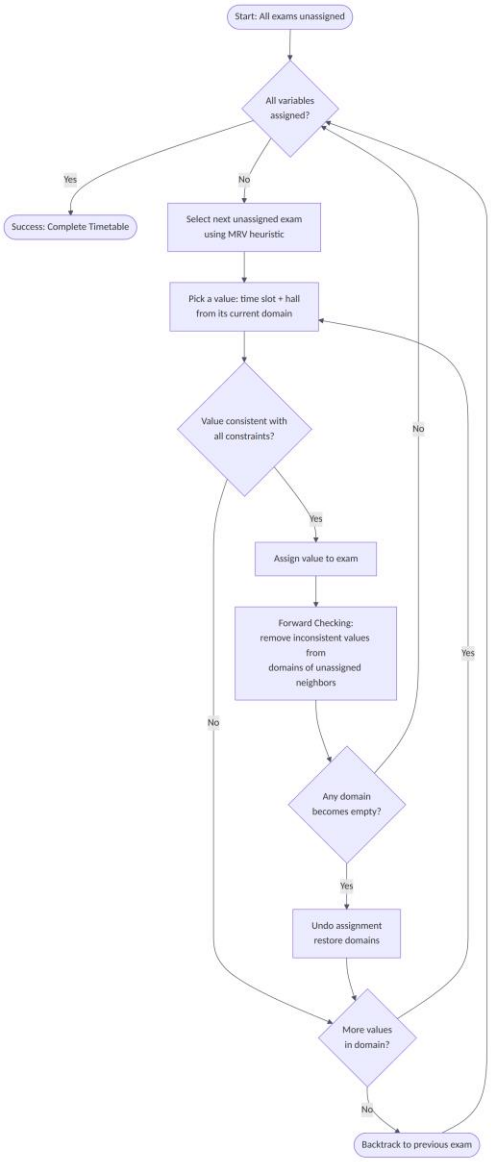
Plain backtracking only discovers a conflict after making a bad assignment several exams later, wasting a great deal of search effort. Forward checking instead immediately removes any value from the domains of not-yet-assigned, constraint-connected exams that would conflict with the exam just assigned. If this removal ever empties another exam's domain completely, the search knows right away — without further exploration

— that the current partial assignment cannot be extended, and it backtracks immediately. This early failure detection dramatically prunes the search tree and is often combined with the MRV heuristic (always expand the exam with the fewest remaining legal values) for further speed-up.

Task 5: Real-World Challenges and Best Strategy for Scalability

Challenges: As course and student counts grow, the constraint graph becomes extremely dense (many students share overlapping course combinations), hall capacity becomes a tight bottleneck, and precedence chains across multiple subjects can create long-range dependencies that are hard to satisfy locally.

Recommended strategy: Backtracking search combined with Forward Checking (or full Arc Consistency via AC-3) and strong variable/value ordering heuristics — Minimum Remaining Values (MRV) to pick the most constrained exam next, and Least Constraining Value (LCV) to try the value that rules out the fewest options for others — offers the best balance of correctness and scalability for university-scale timetabling, since it detects failure early, drastically reduces wasted search, and remains complete (it will find a valid timetable if one exists, or correctly report infeasibility).



Question 5 — Strategic Game AI (Minimax & Alpha-Beta Pruning)

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, and evaluate complex game states with multiple possible moves, while the decision tree is extremely large, making computation expensive.

Task 1: How Minimax Models this Problem

Minimax models the game as a tree where the AI (the maximizing player) and the opponent (the minimizing player) alternate turns. At each level, the maximizing player picks the move leading to the child with the highest evaluated score, while the minimizing player picks the move leading to the child with the lowest score, on the assumption that the opponent always plays optimally against the AI. The value of any state is computed by recursing to a fixed search depth (or terminal state) and propagating evaluation scores back up the tree, so the root move chosen is the one that is best for the AI even under the opponent's best possible counter-play.

Task 2: Computational Challenges of Large Game Trees

- The number of nodes grows exponentially with search depth (branching_factor ^ depth), so exhaustively evaluating a real-time strategy game's tree — which can have dozens of legal moves per turn — quickly becomes computationally infeasible within strict per-turn time limits.
- Memory usage can also grow rapidly if intermediate states or transposition tables are stored naively.
- Because the AI must respond within a hard time budget, plain Minimax often cannot even reach a useful search depth before time runs out, resulting in weak or delayed decisions.

Task 3: How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning adds two bounds while traversing the same Minimax tree: alpha, the best score the maximizer can currently guarantee, and beta, the best score the minimizer can currently guarantee. While exploring a branch, if a node's value falls outside the current [alpha, beta] window — i.e., beta ≤ alpha — the remaining sibling branches at that node can be safely skipped, because a rational opponent would never allow play to reach that branch.

Impact: This produces exactly the same final decision as plain Minimax but explores far fewer nodes; with good move ordering (trying likely-best moves first), Alpha-Beta pruning can reduce the effective branching factor roughly from b to sqrt(b), allowing the AI to search substantially deeper within the same time budget.

Task 4: Designing an Evaluation Function

Because full-depth search is infeasible, the AI evaluates non-terminal states at a cut-off depth using a weighted static evaluation function, typically combining several factors:

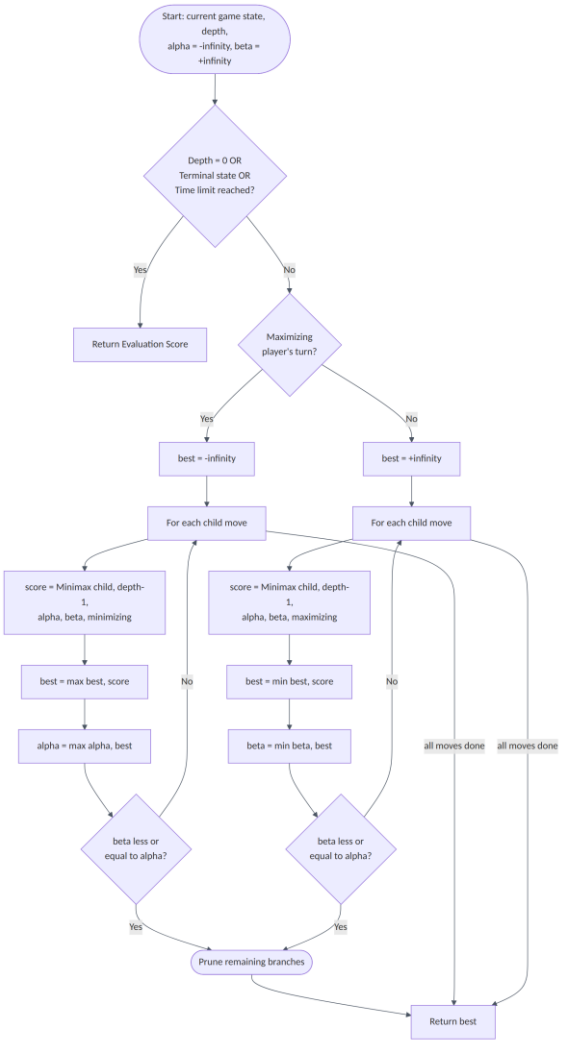
- Material balance — the relative strength/count of each side's units or resources.
- Positional value — control of strategically important map locations (chokepoints, resource nodes, high ground).
- Mobility — the number of legal moves/options available, reflecting flexibility.
- Threat and safety assessment — imminent danger to key units/structures versus offensive opportunities against the opponent.

These factors are combined as a weighted sum (Score = w1·Material + w2·Position + w3·Mobility + w4·Threats), with weights tuned via playtesting or learned from game data, so the function reliably ranks 'good for the AI' states above 'bad for the AI' states even without seeing the game to its end.

Task 5: Strategy for Balancing Optimality and Real-Time Performance

Recommended strategy: Use Alpha-Beta pruning with Iterative Deepening (progressively searching depth 1, then 2, then 3, and so on, always keeping the best result found before the time limit expires), combined with strong move ordering and a well-tuned evaluation function.

Justification: Iterative deepening guarantees the AI always has a usable, complete decision the instant the time budget runs out, since a shallower search finishes first and deeper searches only refine it, directly satisfying the strict real-time constraint. Alpha-Beta pruning maximises how deep that search can go within the time available without altering the correctness of the Minimax decision, and move ordering (e.g., trying previously good moves or captures first) further improves pruning efficiency, together giving the best practical trade-off between decision quality (near-optimal play) and responsiveness.



RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50